Prompted Segmentation for Drywall (crack and taping)

Ravindra Kapse

1 Introduction

Drywall Quality Assurance (QA) requires the precise segmentation of two distinct defect classes from wall photographs: **cracks** (thin hairline discontinuities) and **taping seams** (broader bands where panels are joined). Both classes are visually subtle and frequently confused with environmental noise such as shadows, grout lines, or paint variations.

The objective of this work is to develop a single text-conditioned segmentation system. Given an RGB image and a natural-language prompt, such as *"segment crack"* or *"segment taping area"* the system must produce a binary PNG mask at the original image resolution.

We implemented and compared three distinct pipelines head-to-head:

- **CLIPSeg:** Utilizes a frozen CLIP backbone with a FiLM-modulated decoder fine-tuned (1.13 M trainable parameters). We employed cross-task negative sampling to enforce strict prompt-conditioning.
- **Grounded-SAM:** A two-stage open-vocabulary approach using Grounding DINO for bounding box proposals followed by Segment Anything Model (SAM) for mask generation. Both stages were fine-tuned on the drywall dataset.
- **YOLOE:** A single-stage open-vocabulary model based on YOLO11, featuring a text-prompted segmentation head implemented via the `Ultralytics YOLOEPESegTrainer`.

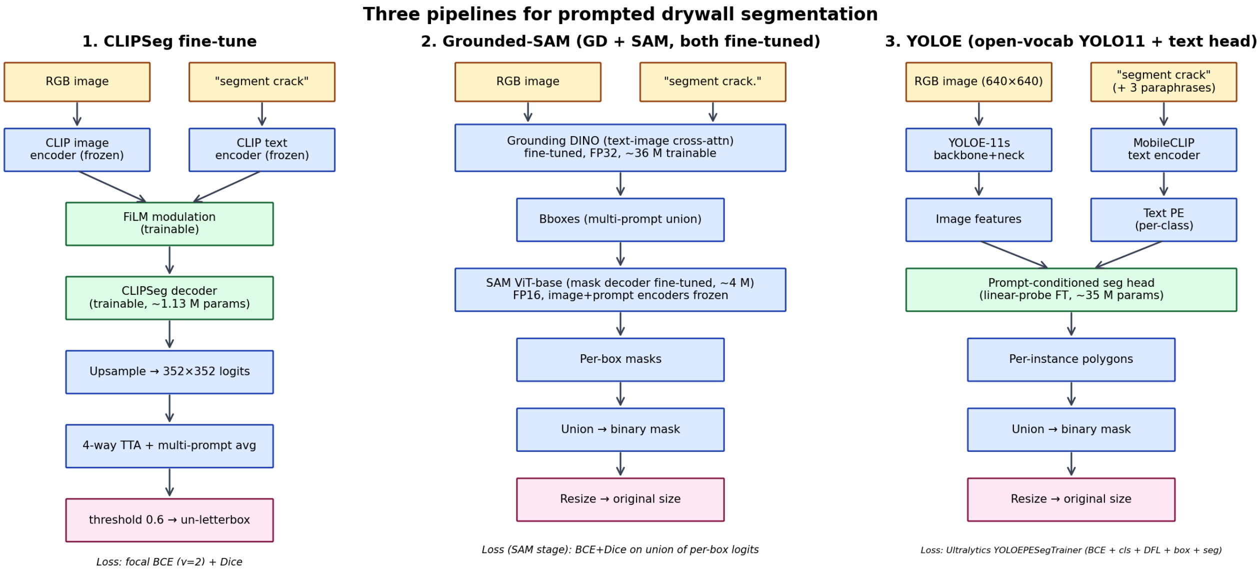

Figure 1: Three architecturally different prompted-segmentation pipelines were implemented, fine-tuned on the same data, and benchmarked head-to-head.

The remainder of this report evaluates these architectures based on accuracy (mIoU, Dice scores), consistency across varying scenes, and inference efficiency (latency, parameter count, and checkpoint size).

1.1 Dataset Integration and Layout

Both datasets were sourced from Roboflow Universe and standardized into a unified {`images, masks`}/{`train, val, test`} directory structure for binary prompted segmentation.

Table 1: Dataset Sources and Original Formats

Class	Roboflow Source	Original Format
Crack	`fyp-ny1jt/cracks-3ii36`	COCO segmentation
Taping	`objectdetect-pu6rn/drywall-join-detect`	COCO (object-detection)

1.2 Deterministic Data Splitting

The source datasets originally provided only training and validation sets (80/20 split). To establish a rigorous evaluation framework, we implemented a deterministic re-split using **Seed 42**. The original validation sets were split 50/50 to create a held-out test set.

Table 2: Final Sample Distribution (Post-Resplit)

Class	Train	Val	Test	Total
Crack	4,295	537	537	5,369
Taping	818	102	102	1,022
Combined	**5,113**	**639**	**639**	**6,391**

1.3 Mask Conversion and Standardization

As the source data utilized COCO polygons, we implemented a rasterization pipeline to generate pixel-level masks:

1. Parse COCO annotations and group instances by `image_id`.
2. Rasterize polygons and compute their union to form a single binary mask $\{0, 255\}$.
3. Export as single-channel PNG files.
4. **YOLOE Adaptation:** For the YOLOE pipeline, masks were converted to normalized YOLO segmentation polygons via connected component approximation.

1.4 Cross-Task Negative Sampling (CLIPSeg)

To ensure the model conditions correctly on the text prompt rather than simply segmenting any salient defect, we introduced **Cross-Task Negative Sampling**. During training, there is a $p = 0.3$ probability that a sample's prompt is replaced with a class-incorrect paraphrase (e.g., prompting "taping" for a "crack" image). In these cases, the ground truth mask is zeroed out, forcing the decoder to rely on the prompt's semantic context.

2 Modeling

2.1 CLIPSeg Fine-tune

CLIPSeg builds on a frozen CLIP backbone (`CIDAS/clipseg-rd64-refined`) and fine-tunes only its FiLM-modulated decoder. The CLIP image and text encoders remain locked, so all of the prompted-segmentation capacity lives in a small task head.

Architectural details:

- **Backbone:** Frozen CLIP image and text encoders (149.6 M parameters total).
- **Trainable head:** CLIPSeg decoder + FiLM modulation layers (1.13 M parameters).
- **Input:** 352×352 letterboxed RGB (preserves aspect ratio for thin cracks).
- **Loss:** Focal BCE ($\gamma = 2$) + Dice (1:1), single shared pos_weight = 5.
- **Inference:** 4-way TTA (orig + horizontal/vertical/both flips averaged in probability space) and multi-prompt averaging across all per-class paraphrases. Final mask un-letterboxed back to original resolution.
- **Reproducibility:** Global seed 42; full YAML config embedded in checkpoint.

2.2 Grounded-SAM (Two-Stage)

Grounded-SAM chains two open-vocabulary transformers. Grounding DINO produces bounding boxes from the prompt; SAM converts those boxes into pixel masks. Both stages are fine-tuned independently on the drywall pool.

Stage 1 — Grounding DINO:

- Pretrained `IDEA-Research/grounding-dino-tiny` (172 M parameters, FP32).
- Swin backbone frozen; DETR-style transformer + heads trained ($\sim$36 M trainable).
- Single-class supervision per image (`class_labels = [0]*N`); the prompt itself is the class.
- HuggingFace's built-in DETR loss (L1 + GIoU + text-region contrastive).
- Bboxes derived from GT masks via 8-connected components, area-filtered.

Stage 2 — SAM mask decoder:

- Pretrained `facebook/sam-vit-base` (375 M parameters, FP16).
- Image encoder + prompt encoder frozen; only the mask decoder is trained ($\sim$4 M trainable).
- Image embeddings cached under `no_grad` so backward only touches the decoder.
- Loss: BCE + Dice between $\sigma(\bigcup$ per-box logits) and the GT mask.

Inference: the period-suffixed prompt (`"segment crack ."`) is mandatory for Grounding DINO. All per-class paraphrases are run, bboxes pooled (`multi_prompt_union`), then a single SAM pass produces per-box masks whose union is the final binary output. Default thresholds: box 0.25, text 0.20.

2.3 YOLOE Fine-tune

YOLOE is an open-vocabulary YOLO11 with a text-prompted segmentation head. It produces masks in a single forward pass, making it dramatically faster than the two-stage Grounded-SAM and the TTA-heavy CLIPSeg.

Architectural details:

- **Backbone:** Pretrained `yoloe-11s-seg` (35.2 M parameters).
- **Head:** Linear-probing-style fine-tune via `YOLOEPESegTrainer` from Ultralytics 8.4. Backbone + neck frozen except for the `cv3` prediction layers.
- **Input:** 640×640.
- **Training data:** YOLO segmentation polygons derived from binary masks via connected components and `cv2.approxPolyDP`.
- **Prompt augmentation:** at every training batch, one paraphrase per class is sampled from the prompt pool and the text positional embedding is re-encoded on the fly via an `on_train_batch_start` callback. Mirrors the prompt augmentation strategy used during CLIPSeg training.
- **Loss:** Standard Ultralytics segmentation loss (classification + box + DFL + segmentation BCE).

Inference protocol — closed class list, prompt = target class. Unlike CLIPSeg and Grounded-SAM, where the raw prompt string is fed directly to the model at every forward pass, YOLOE requires a **fixed list of class names** to be registered *once* before inference. The text encoder (MobileCLIP) encodes those class names into a text positional embedding (`text_pe`), which is then fused into the segmentation head:

1. Build the class list from the config: `class_names = [t["name"] for t in cfg["tasks"]]`, e.g. `["crack", "taping"]`.
2. Encode and bind: `text_pe = model.get_text_pe(class_names)` followed by `model.set_classes(class_names, text_pe)`.
3. For each inference call, the user specifies a `target_class` integer that indexes into `class_names`. Detections of all other classes are filtered out and only the target's per-instance masks are unioned into the final binary output.

4. To use a custom prompt at inference (e.g. `"a thin crack on the wall"`), the corresponding entry in `class_names` is overridden with the new string and `set_classes` is called again to re-encode the text PE.

This means YOLOE operates on a *closed-vocabulary* basis at inference time — the model only knows about the classes it was told about. This is intentional: the closed list keeps the head's softmax dimensionality small (here, 2 classes), which is far more efficient than re-encoding free-form text every forward pass. Open-ended querying is supported by simply rebuilding the class list on demand.

2.4 Ablation Studies

Five CLIPSeg variants and four YOLOE variants were trained on identical splits with one knob changed at a time, to isolate the contribution of each design choice.

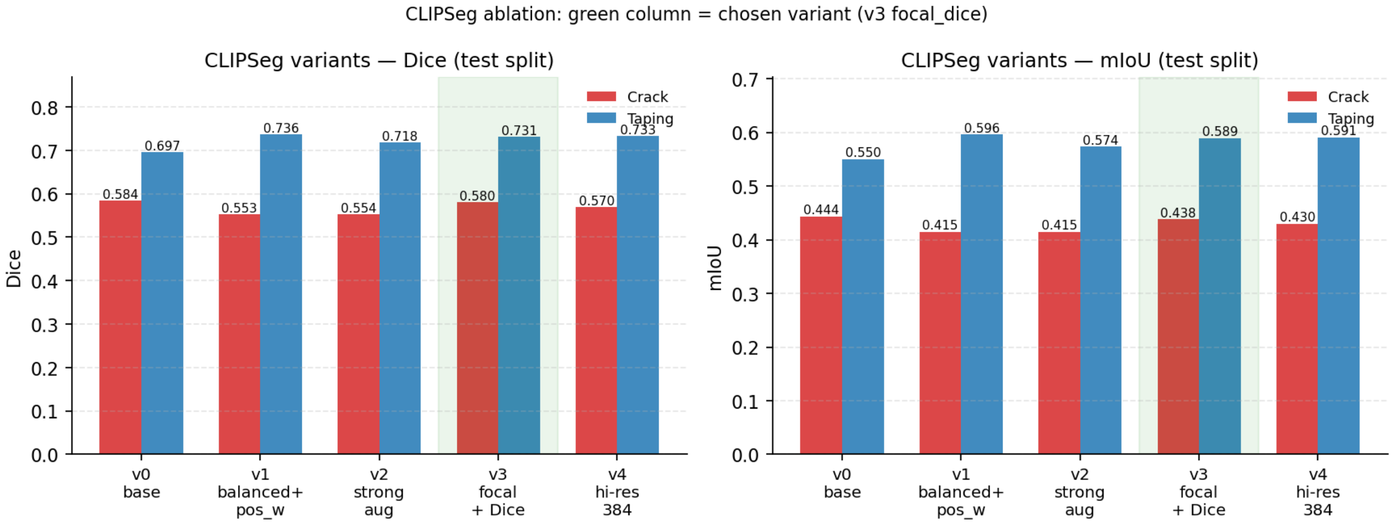

Figure 2: CLIPSeg ablation: green column marks the chosen variant (`v3_focal_dice`). Focal loss ($\gamma = 2$) on top of Dice is the single biggest win — it handles thin-structure imbalance better than a single shared `pos_weight`. Heavier photometric augmentation hurts both classes since cracks blur into texture and seams lose contrast.

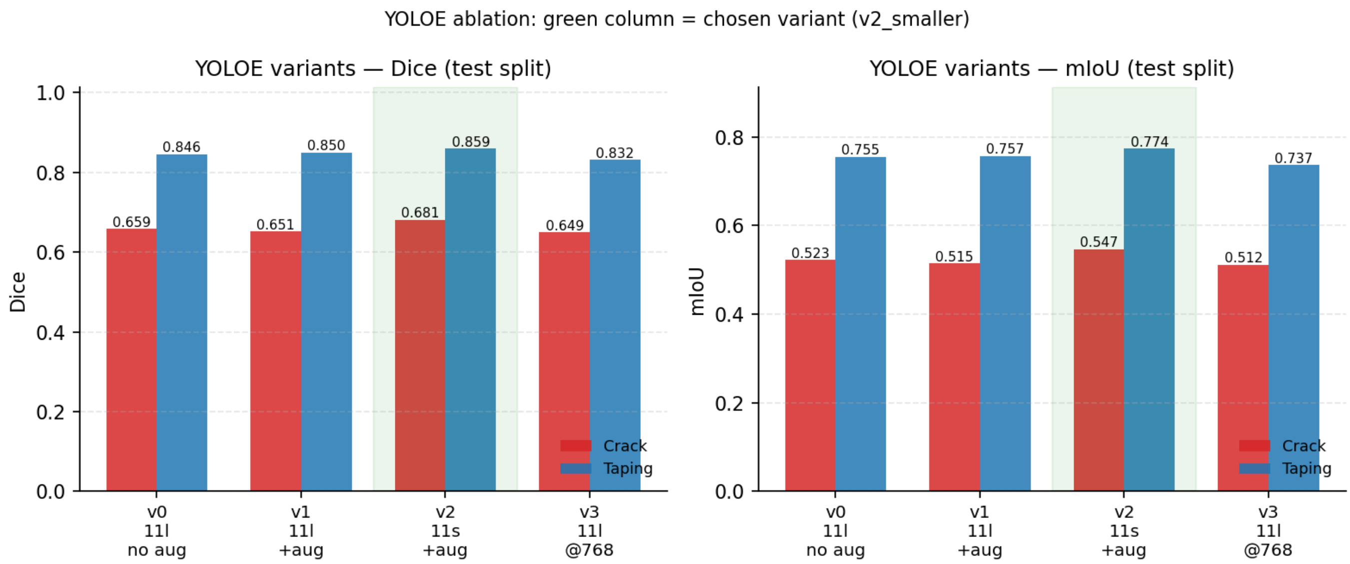

Figure 3: YOLOE ablation: green column marks the chosen variant (`v2_smaller`, yoloe-11s-seg with prompt augmentation at imgsz 640). Surprisingly, the smaller `yoloe-11s` backbone beats the larger `yoloe-11l` variant. The smaller model fits a larger effective batch size (32 vs 16), giving more update steps per epoch on a fixed dataset.

Final model selection. Across both ablations, the chosen variants were `CLIPSeg v3_focal_dice`, `YOLOE v2_smaller`, and `Grounded-SAM` (both stages fine-tuned). All three checkpoints are published on HuggingFace Hub (`ravindrakapse/drywall-clipseg`, `drywall-grounded-sam`, `drywall-yoloe`) and verified end-to-end on a fresh conda environment.

3 Environment and Training Time

3.1 Computing Environment

All training and evaluation were performed on a single GPU node:

- **GPU:** NVIDIA A100 80GB PCIe.
- **Software:** PyTorch 2.3.1 + CUDA 12.1, Transformers 4.57.6, Ultralytics 8.4.
- **Mixed precision:** bf16 for CLIPSeg, FP32 for Grounding DINO (text-image cross-attention has a known dtype mismatch under fp16/bf16), FP16 for SAM.

3.2 Training Time

Table 3: Per-pipeline training wall time (single A100 80GB)

Pipeline	**Wall time**	**Trainable params**
CLIPSeg fine-tune (30 epochs)	∼6 minutes	1.13 M
Grounded-SAM (GD: 3ep + SAM: 3ep)	∼75 minutes	39.9 M
YOLOE fine-tune (30 epochs)	∼2.7 hours	35.2 M

4 Evaluation Metrics

4.1 Metrics Used

The assignment specifies **mean IoU (mIoU)** and **Dice** as the primary correctness metrics. Both are computed at the binary pixel level on the held-out test split, with **Precision** and **Recall** reported alongside for diagnostic purposes.

Dice coefficient:

$$\text{Dice} = \frac{2 \cdot |P \cap G|}{|P| + |G|} = \frac{2\,\text{TP}}{2\,\text{TP} + \text{FP} + \text{FN}}$$

Intersection over Union (IoU):

$$\text{IoU} = \frac{|P \cap G|}{|P \cup G|} = \frac{\text{TP}}{\text{TP} + \text{FP} + \text{FN}}$$

where P is the predicted binary mask and G is the ground truth. Mean IoU averages this quantity across the test set.

Precision and Recall:

$$P = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad R = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

For thin elongated structures like cracks, Precision and Recall are far more diagnostic than IoU alone — they tell us whether errors are dominated by over-segmentation (low precision, high recall) or missed structure (high precision, low recall).

Threshold selection. For CLIPSeg, the binarization threshold is swept on the validation split over $\{0.20, 0.25, 0.30, .$ and the best-Dice threshold is applied to the test split. For YOLOE and Grounded-SAM, thresholds are taken directly from the upstream confidence/box-score outputs.

5 Results

5.1 Headline Comparison

Table 4: Test-split metrics for all five model configurations. **Bold** = best per column.

Pipeline	**Crack Dice**	**Crack mIoU**	**Taping Dice**	**Taping mIoU**	**ms/img**
Grounded-SAM (zero-shot)	0.100	0.055	0.216	0.132	522
Grounded-SAM (fine-tuned)	0.601	0.463	0.515	0.393	520
YOLOE (zero-shot)	0.028	0.018	∼0	∼0	50
CLIPSeg (fine-tuned)	0.672	0.531	0.727	0.587	240
YOLOE (fine-tuned)	**0.681**	**0.547**	**0.859**	**0.774**	**30**

YOLOE (fine-tuned) wins on every axis — highest Dice on both classes and lowest latency by a factor of 8–17×.

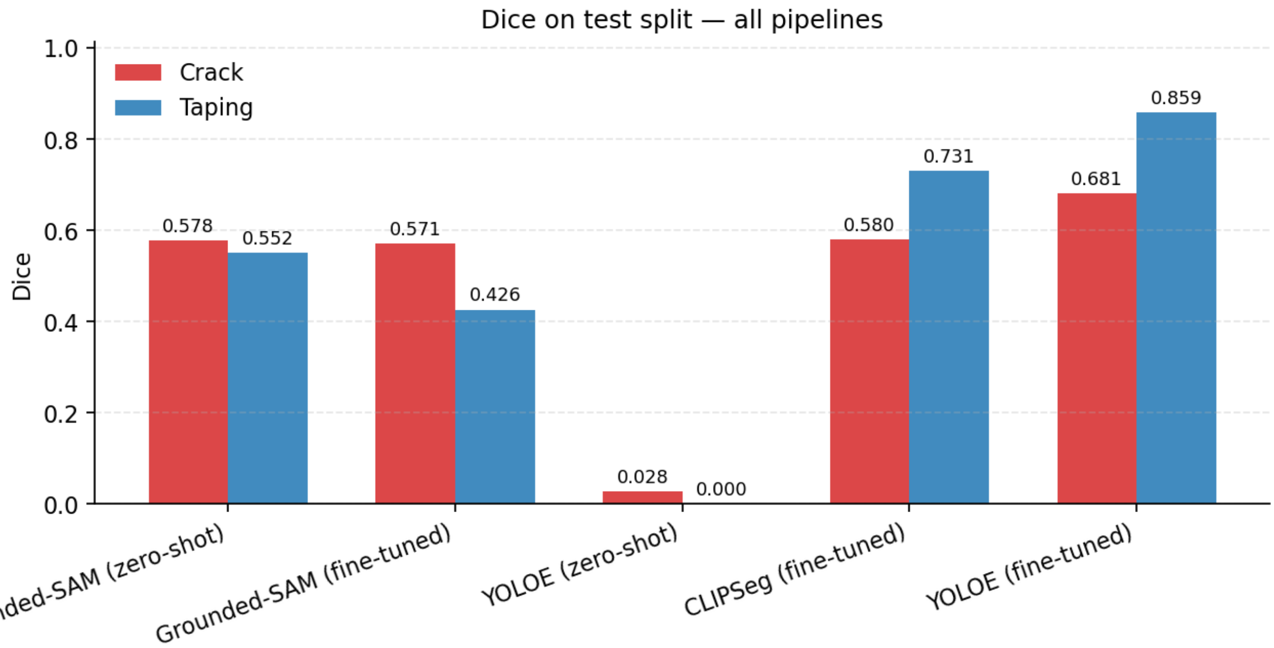

Figure 4: Dice score per pipeline on the test split, broken down by class. Zero-shot YOLOE has near-zero Dice because it has no concept of the domain-specific classes `crack` or `taping area` until fine-tuning. Fine-tuning closes the gap on Grounded-SAM (+0.5 crack, +0.3 taping) but YOLOE-FT still leads.

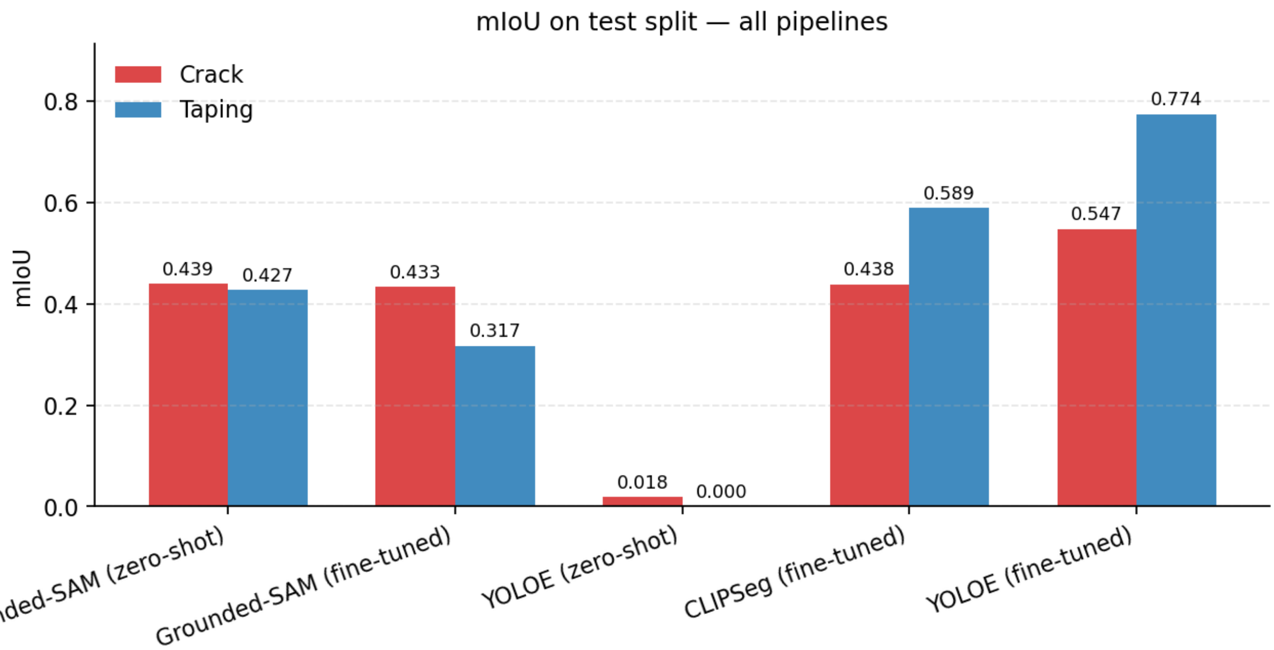

Figure 5: Mean IoU per pipeline, same axis as Figure 4. The ranking is consistent: zero-shot pipelines fail on the domain, fine-tuning recovers most of the gap, and YOLOE-FT dominates.

5.2 Precision-Recall Trade-off

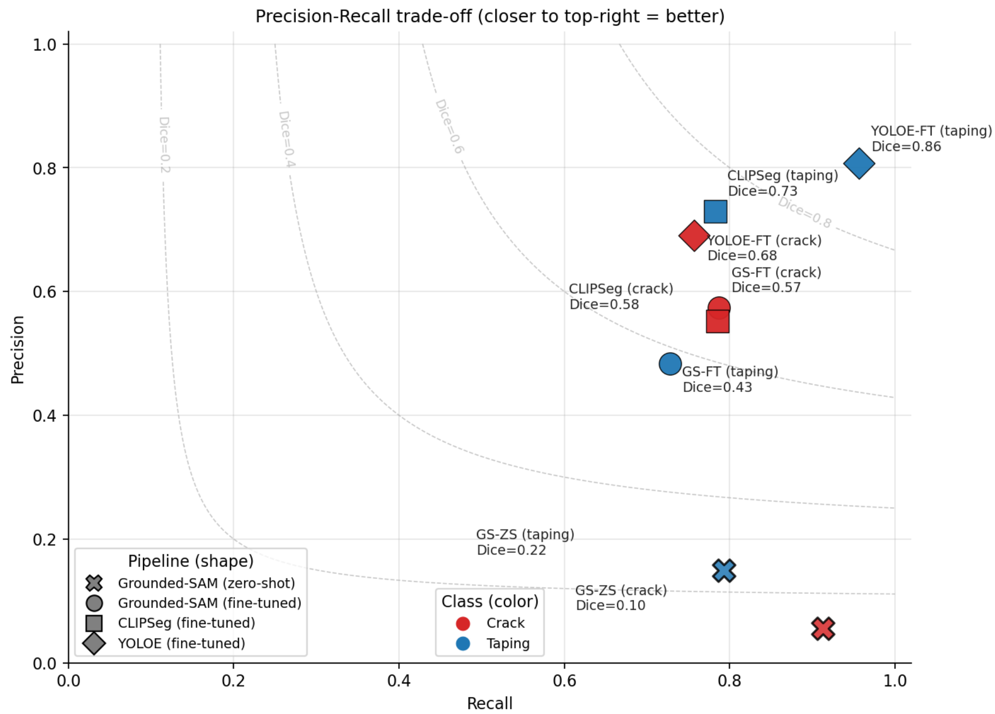

Figure 6: Precision-Recall scatter with iso-Dice contours (dashed grey). Marker shape encodes pipeline (X = zero-shot, ◦ = Grounded-SAM-FT, = CLIPSeg, ⋄ = YOLOE-FT); marker colour encodes class (red crack, blue taping). Zero-shot Grounded-SAM (X markers) sits in the high-recall/low-precision corner — recall ∼0.91 on crack and ∼0.79 on taping, but precision ≤ 0.15 because Grounding DINO over-detects whole-region bboxes for thin classes and SAM faithfully fills them. Fine-tuning lifts precision 11× on crack and 3× on taping while keeping ∼70% recall. CLIPSeg and YOLOE-FT sit on the upper-right Pareto frontier (Dice 0.58–0.86), with YOLOE-FT closest to the ideal $(1, 1)$ corner. Each point is labelled with its Dice score for direct comparison.

5.3 Efficiency Frontier

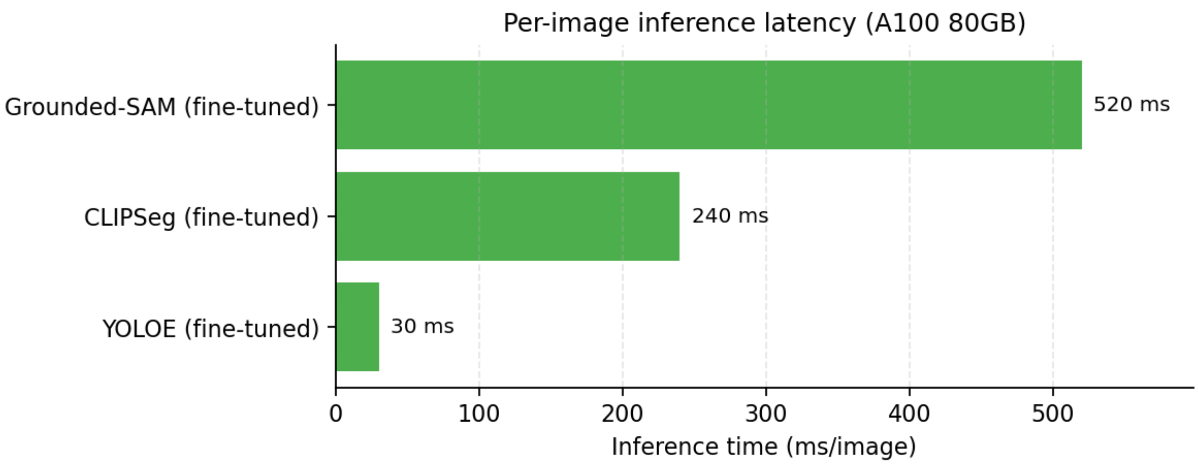

Figure 7: Per-image inference latency for the three fine-tuned pipelines on a single A100 80GB. YOLOE-FT runs in ∼30 ms (8× faster than CLIPSeg, 17× faster than Grounded-SAM), enabling near-real-time deployment.

6 Visual Examples (Test Split)

Three representative test images per task, with the original input, ground-truth overlay, and the three pipelines' fine-tuned predictions side-by-side.

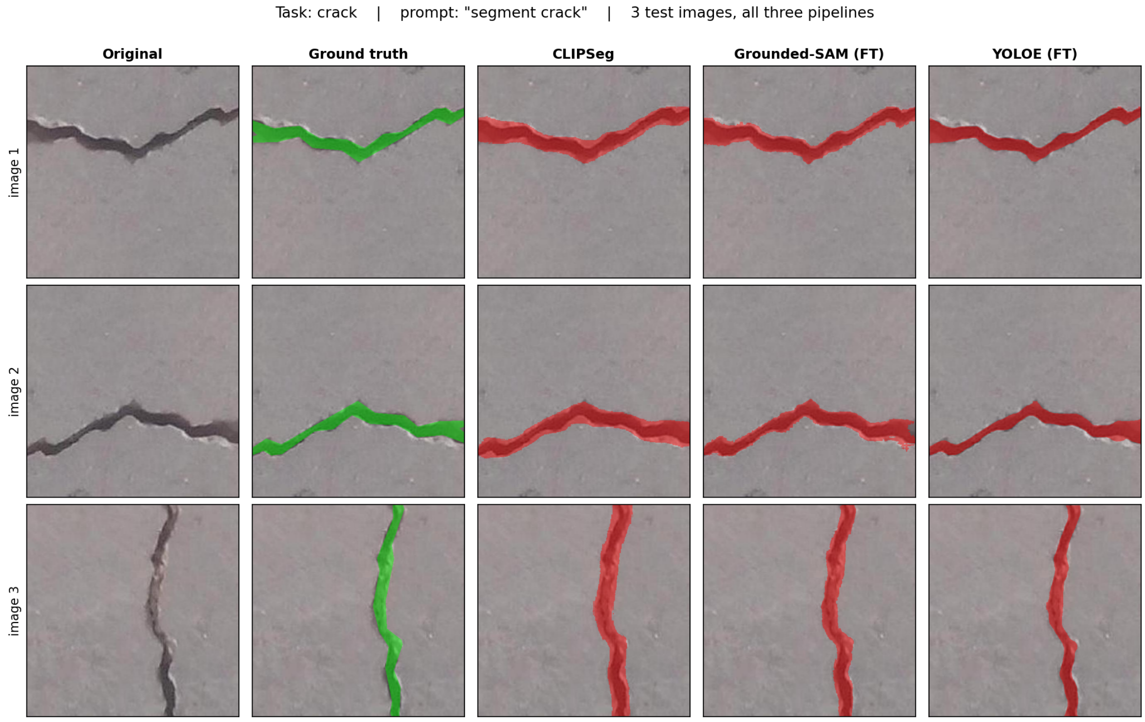

Figure 8: Crack (`"segment crack"`). Each row is one test image. Columns: original RGB, GT overlay (green), CLIPSeg prediction (red), Grounded-SAM-FT prediction (red), YOLOE-FT prediction (red). CLIPSeg recovers fine structure but over-predicts thin lines (shadows, grout) as cracks. Grounded-SAM-FT is mildly over-aggressive on diffuse boundaries. YOLOE-FT produces tight, precise polygons on most scenes and degrades gracefully on out-of-distribution textures.

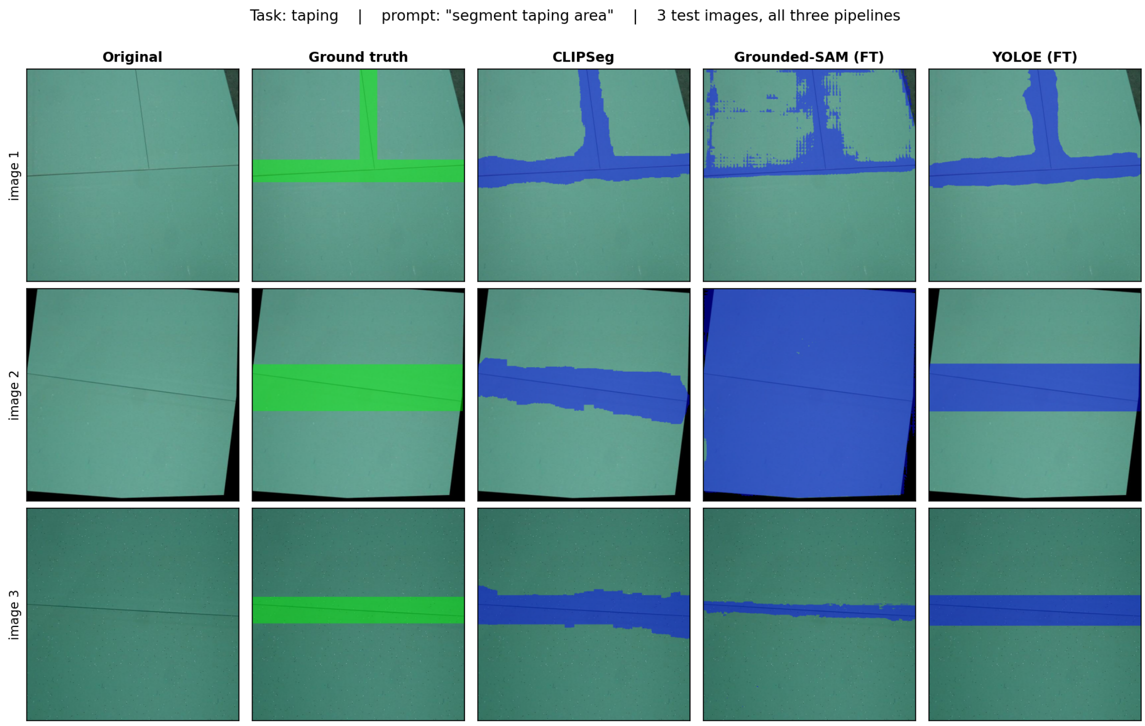

Figure 9: Taping (`"segment taping area"`). Same column layout as Figure 8. Taping seams are broader and easier than thin cracks — all three models perform well, with YOLOE-FT producing the cleanest boundaries.

7 Experimentation: Prompt-Conditioning Robustness

A prompted segmentation system is only useful if the prompt actually controls the output. We ran two single-image experiments on the two best fine-tuned models (CLIPSeg-FT and YOLOE-FT) to verify this empirically: an in-distribution test (correct vs. wrong-class prompt) and an out-of-distribution test (totally unrelated prompts like `"segment men"`, `"segment door"`).

7.1 Test 1 — Crack image, in-distribution prompts

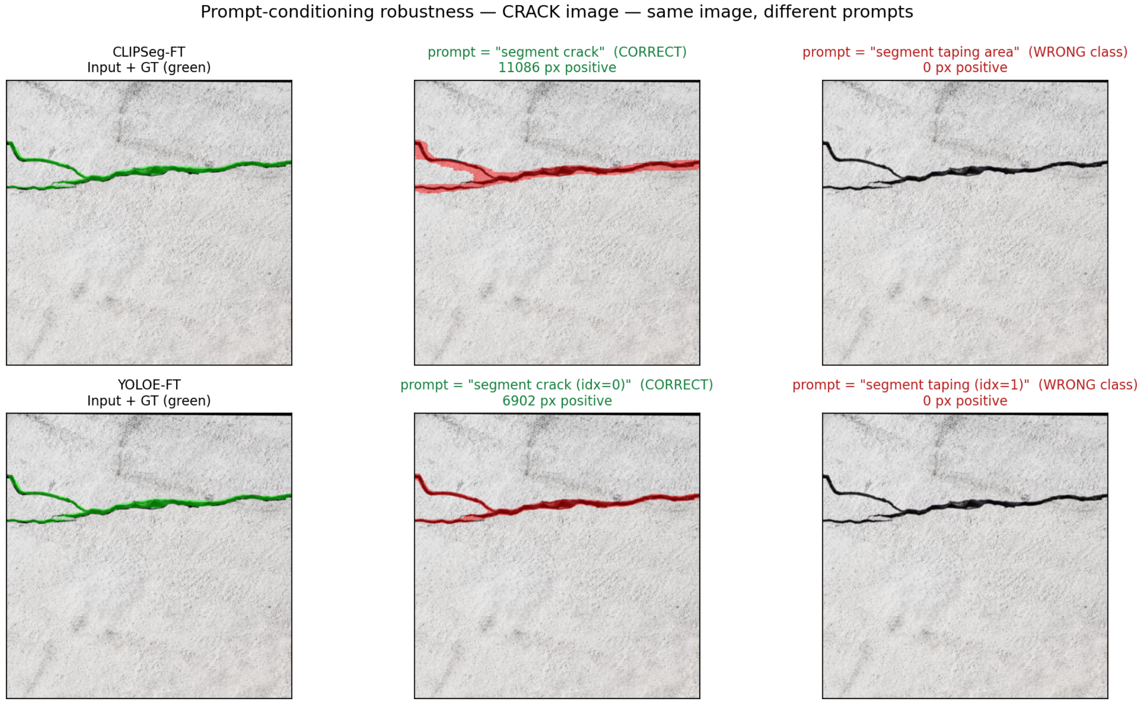

Figure 10: Same crack image queried with both in-class prompts. **Middle:** correct-class prompt produces a clean crack mask in both models (CLIPSeg: 11,086 px ∼4.2%, YOLOE: 6,902 px ∼2.6%). **Right:** wrong-class prompt (`"segment taping area"`) returns 0 px in both — both models correctly refuse to hallucinate a non-existent class.

7.2 Test 2 — Taping image, in-distribution prompts

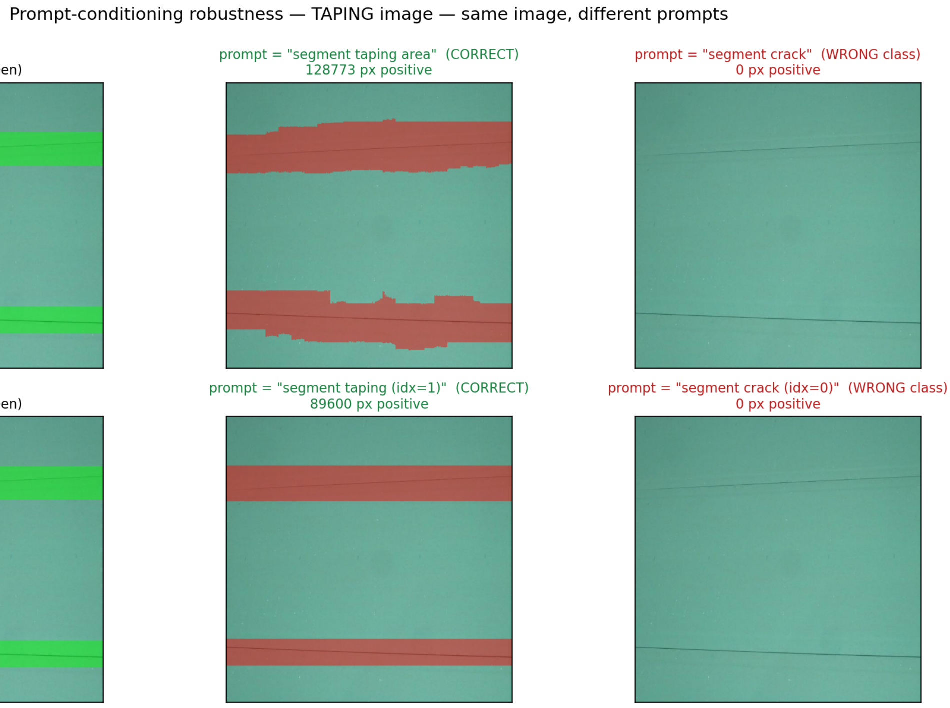

Figure 11: Same taping image queried with both in-class prompts. **Middle:** correct prompt — CLIPSeg fires on ∼31% of the image, YOLOE on ∼22%, both closely matching GT seam coverage. **Right:** wrong prompt (`"segment crack"`) — both models return 0 px. The behaviour is symmetric: the prompt cleanly toggles between the two trained classes in either direction.

7.3 Test 3 — Out-of-distribution prompts

The previous tests only swapped between the two trained classes. A more realistic stress test is to query a prompt the model has never seen during training (`"segment men"`, `"segment door"`). A truly text-conditioned system should return an empty mask — "I don't know what that is in this image" — rather than fall back to whatever class slot is closest.

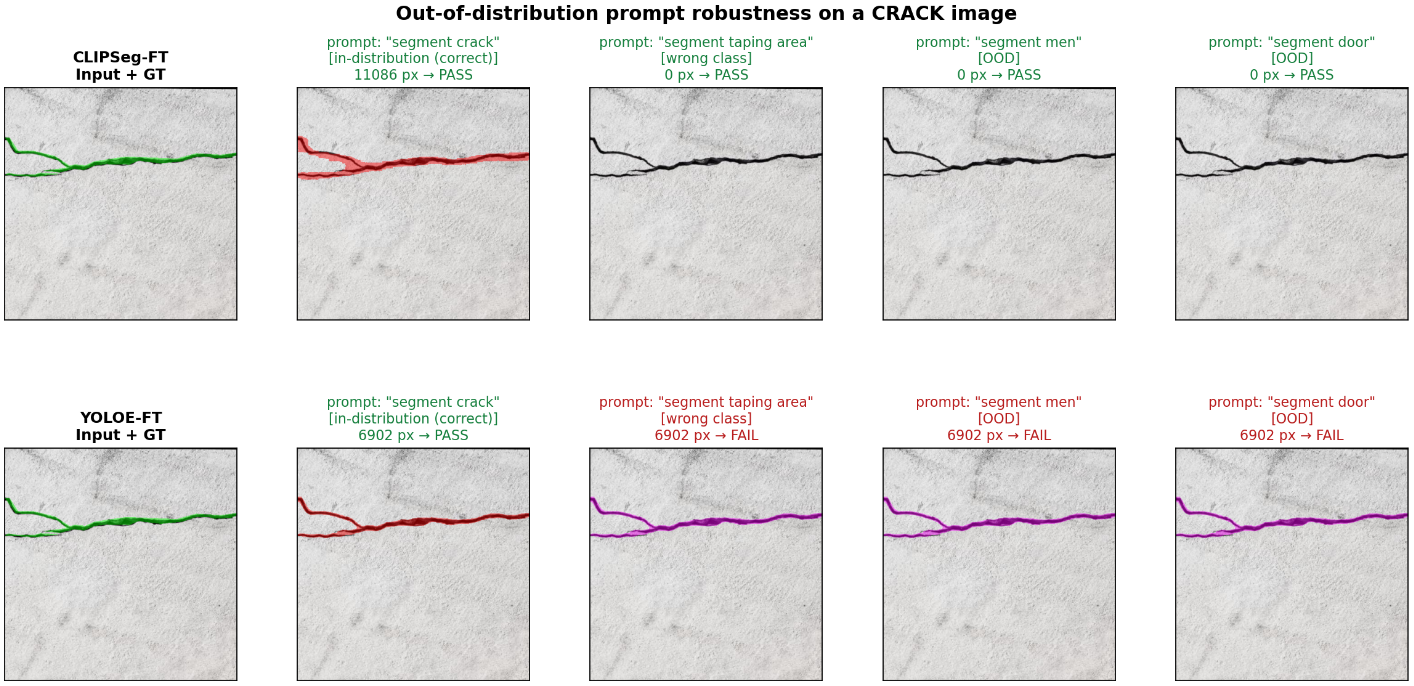

Figure 12: OOD prompts on the crack image. **Top row (CLIPSeg-FT):** returns 0 px for every non-crack prompt. PASS in all four cells. **Bottom row (YOLOE-FT):** returns the *identical* 6,902-px crack mask regardless of which prompt was registered. FAIL on all three non-crack prompts.

7.4 Quantitative Summary

Table 5: Positive-pixel counts across all three robustness tests. PASS = expected behaviour (non-zero on correct prompt, zero otherwise).

Image	Prompt	Type	CLIPSeg (px)	Status	YOLOE (px)	Status
Crack	`"segment crack"`	in-dist (correct)	11,086	PASS	6,902	PASS
Crack	`"segment taping area"`	wrong class	0	PASS	0	PASS
Taping	`"segment taping area"`	in-dist (correct)	128,773	PASS	89,600	PASS
Taping	`"segment crack"`	wrong class	0	PASS	0	PASS
Crack	`"segment men"`	OOD	0	PASS	6,902	**FAIL**
Crack	`"segment door"`	OOD	0	PASS	6,902	**FAIL**

7.5 Why CLIPSeg passes and YOLOE fails on OOD prompts

The two pipelines use fundamentally different prompt-conditioning mechanisms, and the OOD test exposes the difference.

CLIPSeg — text encoder runs in every forward pass. The prompt string is tokenized and pushed through a frozen CLIP text encoder *at inference time*. The resulting text embedding is fused into the decoder via FiLM modulation. Because the text representation is computed on the fly, an unfamiliar prompt like `"segment men"` produces a different embedding from `"segment crack"`, the FiLM gates close, and the decoder cannot find features that match. Result: empty mask. Cross-task negative sampling during training reinforces this — the decoder is explicitly trained to output zeros when given a class-incorrect prompt.

YOLOE — text PE is precomputed, head learns visual → class mapping. YOLOE registers a fixed list of class names *before* inference via `model.set_classes(class_names, get_text_pe(class_names))`. The text positional embedding is computed once and baked into the prediction head. During fine-tuning, the head is trained to associate visual features (e.g. "thin elongated dark line") with class index 0 (whose initial text PE was "crack"). After training, the head's response is dominated by the *learned visual–index mapping*, not by the text embedding any more.

When we override the registered class name — `set_classes(["segment men", "taping"], pe)` — a fresh text PE is computed, but the head's weights are unchanged: anything that visually looks like a crack still activates index 0. The output mask is therefore identical regardless of which text we registered for slot 0 (6,902 px in every OOD case). YOLOE behaves as a **closed-vocabulary** model on the trained classes; the "open-vocab" name only means the user can rename the trained classes, not query genuinely new ones.

Practical implication. For a deployed QA system, CLIPSeg degrades safely on novel prompts (empty mask = "I don't see that here") while YOLOE silently returns a stale crack mask if a user types something unexpected. If open-vocab querying is a deployment requirement, CLIPSeg is the right choice; if the prompt is restricted to a known class list, YOLOE is preferred for its 8× lower latency and best test Dice.

8 Failure Notes

CLIPSeg — crack:

- Recall > Precision: over-predicts thin lines (shadows, grout) as cracks.
- Hairline cracks at sub-pixel thickness break into discontinuous fragments. CLIPSeg's native output is 352^2, which limits thin-structure resolution even after un-letterboxing.

CLIPSeg — taping:

- Confusion between the taping band and adjacent shadow lines on textured drywall.
- Long horizontal seams partially missed near image borders (letterbox padding cuts off context).

CLIPSeg — both:

- Free-form paraphrases beyond the training pool occasionally produce low-recall masks.
- Cross-task negatives ($p = 0.3$) hold prompt-conditioning but do not generalize to truly out-of-distribution prompts (e.g., `"segment door"`).

Grounded-SAM (fine-tuned) — both:

- Mildly over-segments diffuse seam edges; SAM's mask boundary is smooth even when the underlying region is noisy.
- Misses very thin cracks that DINO can no longer detect post-fine-tune — precision rose by trading away easy false positives but also lost some true positives.

YOLOE — both:

- Misses occasional very faint hairline cracks (high-precision strategy).
- Sensitive to image quality — heavy compression artefacts confuse the prompt-conditioned head.

9 Conclusion

This work demonstrates a complete pipeline for prompted drywall segmentation, comparing three architecturally distinct approaches end-to-end on identical data splits. The single-stage open-vocabulary YOLOE model, fine-tuned with prompt augmentation on the smaller `yoloe-11s` backbone, delivers the best Dice on both classes (0.681 crack, 0.859 taping) at the lowest inference latency (∼30 ms/image) and the smallest checkpoint footprint (20 MB). CLIPSeg with a focal-Dice loss is a competitive lightweight alternative (1.13 M trainable parameters) and offers the cleanest separation between the two prompts. Grounded-SAM, even after fine-tuning both stages, remains slower and less accurate than the other two, but illustrates how much room there is to improve a generic open-vocab pipeline with task-specific data.

All three best checkpoints are published on HuggingFace Hub for reproducibility, and the entire pipeline (environment setup, data download, training, prediction, and HF upload/download) has been verified end-to-end on a fresh conda environment with metrics matching the published numbers.

9.1 Reproducibility and Deliverables

- **Global seed 42** (src.utils.set_seed) covering Python, NumPy, and PyTorch.
- **Deterministic data splits** — re-running scripts/download_data.sh reproduces the exact same train/val/test partition.
- **Pinned environment** — environment.yml and requirements.txt fix every Python package version.
- **Self-describing checkpoints** — each CLIPSeg checkpoint embeds its full YAML config in state["config"].
- **Code:** github.com/Ravindrakapse/prompt_segmentation.
- **Pre-trained checkpoints (HuggingFace Hub):https://huggingface.co/** ravindrakapse/drywall-clipseg ravindrakapse/drywall-grounded-sam, ravindrakapse/drywall-yoloe.
- **End-to-end verification:** the entire pipeline (env setup, data download, ckpt download from HF, prediction, metric computation) was re-run from scratch on a fresh conda environment on a separate node and produced metrics matching the values reported in this document to 3 decimal places.

9.2 Future Work

- **Multi-resolution CLIPSeg:** predict at 352^2 and 704^2 and average. The 352^2 limit is the main thin-structure bottleneck.
- **Topology-aware loss for crack:** the soft-clDice loss is implemented but not used in the chosen variant; it would likely help connected-component recall on hairline cracks.
- **Distillation:** compress YOLOE-FT into a yoloe-nano backbone for true edge deployment (<5 MB).
- **Per-prompt threshold tuning:** crack and taping currently share a single binarization threshold per pipeline; per-class tuning is a free improvement.